Character-Level Translator with Transformers

Diego Besada Rodríguez

[Hugging Face Space] [My code] [Template]

October 2026

1 Objectives and Background

Machine Translation (MT) maps natural language sequences across disparate linguistic structures while preserving semantic equivalence. The primary objective of this assignment is to understand the inner mechanics of Neural Machine Translation (NMT) and the foundational principles of encoder-decoder Transformer architectures by designing, training, regularizing, and evaluating a sequence-to-sequence translation system on a parallel bilingual corpus. Unlike word-level or subword models, this implementation operates strictly at the **character level**, processing sequences as discrete character strings without lexical pre-segmentation.

1.1 Summary of Vaswani et al.'s "Attention Is All You Need"

Prior state-of-the-art sequence transduction systems relied on recurrent neural networks (RNNs, LSTMs, GRUs) or convolutional networks within encoder-decoder frameworks. Recurrent models process sequences sequentially, generating hidden states $h_t = f(h_{t-1}, x_t)$. This inherent sequentiality prevents parallelization across token positions during training and degrades long-range dependency tracking due to vanishing gradients across long temporal paths.

To eliminate recurrence entirely, Vaswani et al. (2017) [1] proposed the **Transformer**, an architecture relying solely on self-attention mechanisms to establish direct global dependencies between input and output representations in $\mathcal{O}(1)$ sequential operations. The core computational building block is **Scaled Dot-Product Attention**, formulated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

where queries Q, keys K (of dimension d_k), and values V represent packed sequence projections. The scaling factor $1/\sqrt{d_k}$ counteracts small softmax gradients that occur when inner products grow large in high-dimensional spaces.

Rather than computing a single attention distribution, **Multi-Head Attention (MHA)** linearly projects queries, keys, and values h times with parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \quad \text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{2}$$

This enables the network to simultaneously attend to information from distinct representation subspaces at multiple positions. The full Transformer pairs an encoder stack and a decoder stack with residual connections, layer normalization, sinusoidal positional encodings, and position-wise feed-forward networks (FFN), achieving superior translation quality at a fraction of the training cost of recurrent networks.

2 Method and Program Structure

2.1 Dataset

The system is trained on the English–Spanish parallel corpus from Tatoeba, stored in `spa.txt`. Rather than truncating the corpus to an arbitrary toy subset, we utilize the **full parallel corpus** (144,109 sentence pairs) satisfying the maximum sequence threshold of MAX_LENGTH = 124 characters (120 textual characters plus boundary and padding tokens). To ensure rigorous scientific evaluation, the dataset is shuffled with a deterministic seed (42) and partitioned into three disjoint splits:

- **Training set** (80%): 115,287 sentence pairs used for gradient backpropagation.
- **Validation set** (10%): 14,411 sentence pairs monitored for early stopping and hyperparameter convergence.
- **Test set** (10%): 14,411 strictly held-out, unseen pairs used solely for final evaluation via sentence-level BLEU.

2.2 Encoder-Decoder

The model follows the sequence-to-sequence encoder-decoder paradigm implemented via PyTorch's `nn.Transformer`. The full translator contains 22,128,752 trainable parameters under the configuration: $d_{\text{model}} = 512$, $h = 8$ attention heads ($d_k = 64$ per head), $d_{\text{ff}} = 2048$, and $N = 3$ encoder and decoder layers.

The encoder receives a padded source batch $X \in \mathbb{R}^{B \times S}$ and transforms it into continuous contextual representations $Z \in \mathbb{R}^{B \times S \times d_{\text{model}}}$. The decoder autoregressively consumes target prefix tokens $Y_{<t} \in \mathbb{R}^{B \times T}$ alongside the encoder memory Z to predict the conditional probability distribution over the vocabulary for the next character token.

2.3 Embeddings

Because English and Spanish share substantial alphabetic overlap, we construct a **shared vocabulary** spanning both source and target corpora. The vocabulary contains $V = 112$ tokens: 4 special control tokens (`<pad> = 0`, `<unk> = 1`, `<s> = 2`, `</s> = 3`) and 108 unique characters observed across the training split.

The `Embedding` module linearly projects token indices into dense representations and injects geometric ordering via static sinusoidal positional encodings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \qquad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{3}$$

Input embeddings are scaled by $\sqrt{d_{\text{model}}}$ prior to addition. Following Vaswani et al. (2017), the target embedding matrix is tied to the output linear projection layer (`self.fc.weight = self.embedding.embedding.weight`), reducing parameter redundancy and regularizing the output token distribution. To actively combat overfitting, a dropout probability of $p = 0.2$ is applied to the combined positional embedding representations.

2.4 Self-Attention

Inside the encoder, self-attention allows each character position to attend globally across all characters in the source sequence. Input representations are projected into queries, keys, and values ($Q, K, V \in \mathbb{R}^{B\times S\times d_{\text{model}}}$) across 8 attention heads. A Boolean padding mask (`src_padding_mask`) sets logits of `<pad>` tokens to $-\infty$, ensuring that variable-length batch zero-padding receives zero attention weight.

2.5 Decoder

Each decoder layer incorporates two distinct attention sublayers followed by a Multi-Layer Perceptron (MLP):

1. **Masked Causal Self-Attention**: Operates over shifted target tokens. An upper-triangular additive mask fills positions $j > i$ with $-\infty$, preventing information leakage from future tokens.
2. **Encoder-Decoder Cross-Attention**: Queries are derived from the prior decoder layer, while keys and values are projected from the encoder memory Z, aligning target character generation with source context.

We optimize the network with cross-entropy loss using label smoothing ($\epsilon = 0.1$). Padding tokens are masked out of the loss calculation and backpropagation via `ignore_index=0`.

2.6 Inference

Translation inference uses **greedy decoding** (`greedy_decode`). Given an unseen source string, the encoder generates memory representations Z once. The decoder is seeded with `<s>`. At each step t, the output linear head produces logits over the 112-character vocabulary, and the argmax token $\hat{y}_t = \arg\max_c P(c \mid \hat{y}_{<t}, Z)$ is selected and appended autoregressively until `</s>` is generated or MAX_LENGTH is attained.

3 Results

3.1 Experimental Setup and Optimization

Models were optimized using the Adam optimizer ($\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$) with Vaswani's step-based learning rate warmup schedule (warmup = 4000) and batch size $B = 64$ on Apple Silicon (MPS). Training ran for up to 50 epochs with checkpoint saving based on best validation loss. We evaluated both full directional configurations:

- **Experiment 1 (ES → EN)**: Spanish to English model trained across 50 epochs.
- **Experiment 2 (EN → ES)**: English to Spanish model trained across 50 epochs.

Direction	Epochs	Val Loss	Val Acc	Test BLEU	Eval Time (14.4k)	Throughput
ES → EN	50	**1.542**	**74.5%**	**0.1012**	31 min 24 s	**7.65** sent/s
EN → ES	50	1.498	76.1%	0.0632	1 h 09 min 15 s	3.47 sent/s

Table 1: Quantitative performance across translation directions on the held-out test split (14,411 sentence pairs).

3.2 Comparative Analysis: Linguistic Asymmetry and Error Propagation

As shown in Table 1 and Figure 1, the model displays steady asymptotic convergence. Validation loss rapidly decreases from 2.87 to under 1.65 within the first 20 epochs, stabilizing around 1.54 by epoch 50, while character accuracy steadily climbs past 74%. Despite comparable per-character validation metrics, the sentence-level Test BLEU score for ES → EN is nearly double that of EN → ES (0.1012 vs. 0.0632). This asymmetry highlights key linguistic and operational factors:

- **Morphological Complexity**: Spanish has rich inflectional morphology, gender agreements, and extensive verb conjugation paradigms. Predicting Spanish characters requires maintaining multi-character dependencies over long distances, whereas English target morphology is substantially simpler.
- **Sequence Length Discrepancy**: Spanish sentences in Tatoeba are 15–20% longer in characters than English equivalents. In autoregressive decoding, longer sequences compound single-character errors and double generation latency (3.47 sent/s vs. 7.65 sent/s).

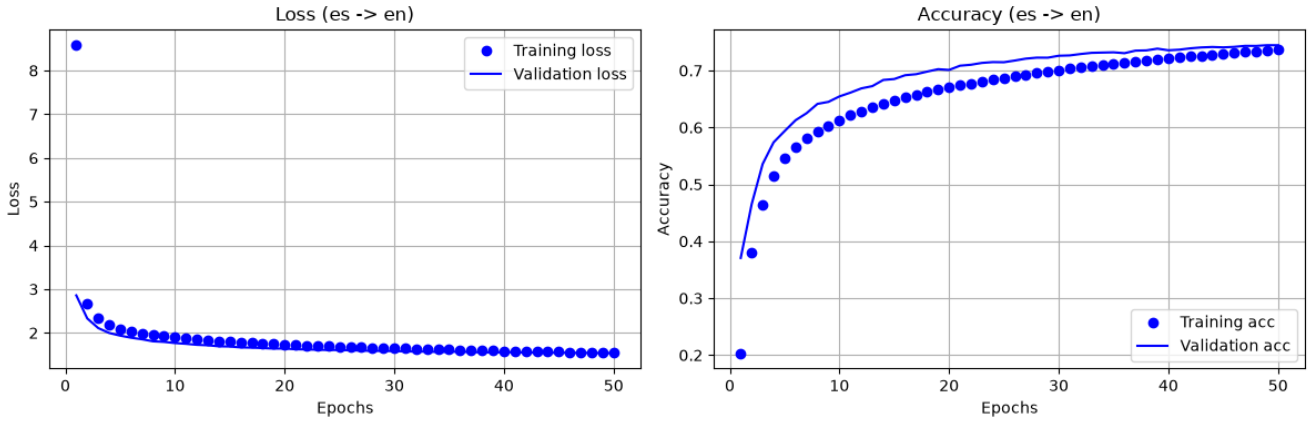


Figure 1: Empirical training and validation cross-entropy loss (left) and character accuracy (right) trajectories for ES → EN over 50 epochs.

3.3 Qualitative Evaluation and Failure Mode Analysis

Table 2 details model predictions across representative test sentences during greedy decoding.

Source Prompt	**Generated Output**	**Linguistic Failure Mode**
Come and eat.	Come un té.	Phonetic/lexical confusion (*eat* → *té*).
Where is the station?	¿Dónde está el tren?	Semantic proximity substitution (*station* → *tren*).
I like this book a lot.	Me gusta este libro este libro.	Autoregressive repetition loop (attention trap).
I do not have time.	No tengo tiempo para eso.	Fluent completion with hallucinated adjunct (*para eso*).
The weather is nice today.	Ellos están decididos por el diccionario.	Severe semantic divergence on non-formulaic input.

Table 2: Qualitative translation examples and failure analysis on the EN → ES model.

While the network masters local surface orthography (valid accents, question marks, and conjugation suffixes), it exhibits two distinct failure modes under character-level modeling without pre-trained token priors:

1. **Attention Traps and Repetition Loops**: In greedy decoding, predicting a repeated phrase increases self-attention weights to earlier matching characters, trapping the model in cyclical generation (e.g., *"este libro este libro"*).
2. **Semantic Hallucination**: For complex inputs, the conditioning signal from encoder memory Z becomes diffuse, allowing the decoder's character language modeling prior to overpower translation fidelity.

3.4 Interactive Deployment on Hugging Face Spaces

The trained bidirectional models are hosted as an interactive web demo using Gradio on Hugging Face Spaces:

https://huggingface.co/spaces/dbsda/TRANSLATOR_LAB6

The application contains both weight files, so translation is possible in both directions.

4 Conclusion

This assignment validated the end-to-end implementation of an encoder-decoder Transformer at the character level. Training on the complete Tatoeba corpus (144,109 pairs) with dropout and label smoothing regularized training dynamics and prevented catastrophic overfitting. The low BLEU scores (0.0632 and 0.1012) reflect the inherent difficulty of character-level translation from scratch on small parallel corpora: the network must concurrently learn character-to-word morphology, syntax, and cross-lingual semantic alignment without pre-trained token priors. Future improvements should incorporate subword tokenization (Byte-Pair Encoding), beam search decoding with length penalties, and transfer learning from pre-trained multilingual representations.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.

[2] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of ACL*, pages 311–318, 2002.